



Rapport du projet front-end
Sous le thème :
Système de Gestion des Profiles

Filière : développement Web

Réalisé par : EL Rhinaoui Adam - Berriq Ali

Année académique : 2025/2026

Abstract

This report presents the development of a web application dedicated to user profile management. The primary objective of this project was to design and implement a centralized, secure, and efficient solution for managing users, roles, and applications, while ensuring better control, flexibility, and organization within information systems.

The developed application is structured around two main dashboards:

An Administrator dashboard, responsible for managing roles and applications through operations such as creation, modification, deletion, and

role assignment, as well as managing users and assigning managerial privileges.

A **Manager dashboard**, designed to facilitate the assignment of roles and applications to standard users according to the configurations previously established by the administrator.

From a technical perspective, the application was developed using **Java Spring Boot** for the backend and **React.js with JavaScript** for the frontend, while **MySQL** was adopted as the database management system to ensure reliable and efficient data storage and processing.

The design and modeling phase was supported by **StarUML** and **PlantUML**, which were used to create the **Conceptual Data Model (CDM/MCD)**, **use case diagrams**, and **class diagrams**.

By following a structured development methodology and a well-defined project life cycle supported by a **Gantt planning approach**, the project was successfully implemented in accordance with both functional and technical requirements. The resulting solution ensures maintainability, scalability, and an effective management process for user profiles and access right

Résumé

Ce rapport présente le développement d'une application web dédiée à la gestion des profils utilisateurs. L'objectif principal de ce projet consiste à concevoir et mettre en œuvre une solution centralisée, sécurisée et performante permettant l'administration des utilisateurs, de leurs rôles ainsi que des applications auxquelles ils peuvent accéder.

L'application développée repose sur une architecture organisée autour de deux tableaux de bord principaux :

- **Le tableau de bord Administrateur**, destiné à la gestion complète des rôles (création, modification et suppression), des applications (création, modification, suppression et association des rôles) ainsi qu'à la gestion des utilisateurs à travers l'attribution du rôle de gestionnaire et des applications associées.
- **Le tableau de bord Gestionnaire**, permettant l'attribution des rôles et des applications aux utilisateurs standards, conformément aux configurations préalablement définies par l'administrateur.

Sur le plan technique, le projet a été développé en utilisant **Spring Boot (Java)** pour la partie backend, **React.js** pour la conception de l'interface utilisateur et **MySQL** comme système de gestion de base de données. La phase de conception et de modélisation a été réalisée à l'aide des outils **StarUML** et **PlantUML**, ayant permis l'élaboration du **Modèle Conceptuel de Données (MCD)**, des **diagrammes de cas d'utilisation** ainsi que des **diagrammes de classes**.

Cette approche a permis de mettre en place une solution répondant aux exigences de sécurité, de flexibilité et de maintenabilité, tout en assurant une gestion efficace des profils utilisateurs et des droits d'accès au sein du système.

Table des figures

2.1 Cycle de vie du projet (V).	21
2.4 Diagramme de Gantt	22
3.5 Modèle conceptuel de données (MCD).	27
4.2 schéma la base de données.	30
4.3 interface admin	31
4.3 interface gestionnaire	32
4.4.1 diagramme de cas d'utilisation	33
4.4.2 diagramme de classe	34
5.1.1 Java spring boot	35
5.1.2 React – javascript	36
5.2 MYSQL workbench	37
5.3 Dashboard Admin – rôle –app-utilisateur	39
5.3 Dashboard Gestionnaire – Attribution des rôles et applications.	41

Table des matières

Abstract	2
Résumé	4
Introduction générale	9
1 Chapitre I : Contexte générale du projet	10
1.1 Introduction	10
1.4 Problématique et importance de la gestion des profils utilisateurs:	12
1.5 Objectifs du Projet :	12
1.6 Conclusion	13
2 Chapitre II : Démarche et Conduite du Projet	14
2.1 Introduction	14
2.2 Méthodologie de conduite du projet :	14
2.3 Cycle de vie du projet (modèle en V) :	15

2.4 Conclusion et Diagramme de Gantt :	17
2.5 Conclusion :	19
3 Chapitre III : Analyse des besoins.....	19
3.1 Introduction :	19
3.2 Recueil des Besoins Fonctionnels :	20
3.3 Recueil des Besoins Non Fonctionnels :	21
3.4 Modélisation des Processus Métiers :	22
3.5 Modèle Conceptuel de Données (MCD) :	23
3.6 Conclusion :	24
4 Chapitre IV : Conception de l'application :	24
4.1 Architecture générale de l'application :	24
4.1.1 Côté Backend :	25
4.1.2 Base de données :	25
4.1.3 Côté Frontend :	25
4.1.4 Sécurité et authentification :	25
4.2 Conception de la base de données (schéma relationnel) :	26
4.3 Conception des interfaces utilisateurs	26
4.4 Diagrammes UML :	28
4.4.1 Diagramme de cas d'utilisation	28
4.4.2 Diagramme de classes	30
4.5 Conclusion	30
5 Chapitre V : Réalisation et mise en œuvre :	31
5.1 Environnement de développement	31
5.1.1 Backend : Spring Boot (Java)	31
5.1.2 Frontend : React.js et JavaScript :	32
5.2 Base de données : MySQL	33
5.2.4 Outils de conception :	34
5.2 Implémentation des modules fonctionnels :	34
5.2.1 Module gestion des rôles :	34
5.2.2 Module gestion des applications :	34
5.2.3 Module gestion des utilisateurs :	34

5.3	Interfaces de l'application :	35
5.5	Conclusion :	39
5.6	Conclusion générale :	39
5.7	Bibliographie :	40

Introduction générale

Dans le cadre de notre formation d'ingénierie à l'École Marocaine des Sciences de l'Ingénieur (EMSI), la réalisation d'un stage de fin d'année constitue une étape essentielle. Il s'agit d'une opportunité privilégiée pour mettre en pratique les connaissances théoriques acquises et les confronter aux réalités du monde professionnel.

C'est dans ce contexte que j'ai intégré l'Administration des Douanes et Impôts Indirects (ADII), au sein de la Division de la Programmation et de la Communication à Rabat, pour une période allant du 29 juillet au 05 septembre 2025.

Le thème qui m'a été confié portait sur le développement d'une application web de gestion des profils utilisateurs. Ce choix s'inscrit dans une problématique concrète : l'ADII gère un nombre important d'applications internes et externes, nécessitant une organisation stricte des rôles et des droits d'accès. L'objectif est donc de proposer une solution moderne, centralisée et sécurisée, permettant d'optimiser la gestion des profils et de renforcer la traçabilité ainsi que la gouvernance des accès.

Le développement de cette application s'est appuyé sur une architecture technique moderne, avec Java Spring Boot pour la partie backend, React.js pour le frontend, et MySQL comme système de gestion de base de données. La conception a été assurée à l'aide d'outils tels que StarUML et PlantUML pour les diagrammes (MCD, diagrammes de cas d'utilisation, diagrammes de classes). Le projet a suivi une méthodologie basée sur le modèle en V,

garantissant une bonne traçabilité entre les étapes de conception, d'implémentation et de validation.

Ce rapport présente donc le cheminement complet du projet, depuis l'étude des besoins jusqu'à la mise en œuvre, en passant par la conception et les phases de test. Il met également en avant les apports pédagogiques et professionnels de cette expérience, qui a été pour moi une véritable occasion de progresser tant sur le plan technique que personnel.

1 Chapitre I : Contexte générale du projet

1.1 Introduction

La réussite d'un projet informatique repose en grande partie sur la compréhension du contexte dans lequel il s'inscrit. Avant de présenter les aspects techniques et fonctionnels du projet de gestion des profils utilisateurs, il est essentiel de situer l'environnement d'accueil ainsi que les motivations qui ont conduit à ce choix. Ce chapitre présente donc une vue d'ensemble de l'Administration des Douanes et Impôts Indirects (ADII), de la division dans laquelle le stage a été effectué, ainsi que la problématique de gestion des profils utilisateurs et les objectifs visés par le projet.

La facilitation et la sécurisation des échanges commerciaux à travers la mise en place de systèmes modernes de contrôle et de dédouanement.

La lutte contre la fraude, la contrebande et les infractions liées au commerce international.

Chapitre I : Contexte générale du projet

L'accompagnement des opérateurs économiques en leur offrant des services numériques innovants.

Grâce à une stratégie de digitalisation avancée, l'ADII a mis en place plusieurs plateformes et systèmes d'information qui permettent d'automatiser et de fluidifier ses opérations. Toutefois,

cette multiplication d'applications génère un besoin croissant en termes de gestion des utilisateurs, des rôles et des droits d'accès.

1.4 Problématique et importance de la gestion des profils utilisateurs:

Avec la multiplication des applications et des services numériques déployés par l'ADII, la gestion des utilisateurs et de leurs droits d'accès devient un enjeu stratégique. Une gestion manuelle ou non centralisée des rôles peut engendrer plusieurs problèmes :

Complexité administrative : difficulté à gérer les multiples profils sur différentes applications.

Manque de sécurité : risque d'attribution erronée de droits ou d'accès non autorisés.

Manque de traçabilité : difficulté à suivre l'historique des attributions et des modifications de rôles.

Perte d'efficacité : temps important consacré par les administrateurs à des tâches répétitives.

Le projet vise donc à répondre à ces problématiques en développant une application web qui centralise la gestion des profils utilisateurs, offre une meilleure visibilité sur les rôles et assure une sécurité renforcée grâce à une architecture bien définie et un processus de déploiement contrôlé.

1.5 Objectifs du Projet :

L'objectif principal du projet est de concevoir et développer une application web moderne et sécurisée de gestion des profils utilisateurs pour les applications internes de l'ADII. Les objectifs spécifiques sont :

Mettre en place un dashboard administrateur permettant la gestion des rôles, des applications et des utilisateurs.

Développer un dashboard gestionnaire facilitant l'attribution des rôles et des applications aux utilisateurs finaux.

Centraliser la gestion des profils afin d'améliorer la traçabilité et la sécurité des accès.

Utiliser une architecture technique moderne reposant sur Java Spring Boot, React.js et MySQL.

Déployer l'application sur l'environnement de production de l'ADII avec Wildfly (JBoss) et Microsoft Active Directory pour l'authentification et la sécurité.

1.6 Conclusion

Ce premier chapitre a permis de présenter le cadre général du projet, en mettant en évidence le rôle et les missions de l'ADII, la fonction de la division d'accueil ainsi que la problématique qui a motivé la mise en place d'une solution de gestion centralisée des profils utilisateurs. Les objectifs fixés définissent clairement la direction à suivre pour répondre aux besoins de l'institution.

Le chapitre suivant portera sur la méthodologie adoptée et la démarche de conduite du projet, notamment à travers le cycle de vie en V et la planification représentée par le diagramme de Gantt.

2 Chapitre II : Démarche et Conduite du Projet

2.1 Introduction

La réussite d'un projet informatique repose en grande partie sur la méthodologie adoptée. Une démarche structurée permet de réduire les risques, d'assurer une meilleure visibilité sur l'avancement et de garantir que le produit final réponde fidèlement aux besoins exprimés.

Dans le cadre de mon projet de stage à l'Administration des Douanes et Impôts Indirects (ADII), il était primordial d'adopter une approche rigoureuse et adaptée au temps imparti, afin d'assurer une progression méthodique depuis l'étude des besoins jusqu'à la mise en production.

Pour cela, j'ai opté pour une méthodologie en V, qui offre un cadre clair, assure la traçabilité des activités et met l'accent sur la validation de chaque étape par des tests correspondants. Cette approche a été complétée par un outil de planification : le diagramme de Gantt, permettant une gestion efficace du temps et des tâches.

2.2 Méthodologie de conduite du projet :

La méthodologie de conduite adoptée se fonde sur trois principes majeurs :

Organisation structurée : chaque étape du projet est planifiée avec des objectifs précis et des livrables définis.

Validation progressive : à chaque phase de conception correspond une phase de test, ce qui permet d'identifier rapidement d'éventuelles incohérences.

Traçabilité complète : grâce au modèle en V, chaque exigence identifiée en amont est reliée à un test en aval, garantissant ainsi la conformité de l'application développée.

Les étapes clés de la méthodologie en V :

Analyse des besoins : identification des besoins fonctionnels et non fonctionnels exprimés par l'ADII.

Spécifications : traduction de ces besoins en spécifications techniques précises.

Conception : élaboration des modèles de données et des diagrammes UML (MCD, Use Case, Diagramme de classes, etc.) pour tracer l'architecture de l'application.

Implémentation : développement de l'application en utilisant Java Spring Boot, MySQL et les outils de conception StarUML / PlantUML.

Tests unitaires : vérification du bon fonctionnement de chaque module indépendamment.

Tests d'intégration : validation de la cohérence entre les modules (base de données, interfaces utilisateurs, logique métier).

Recette et validation : comparaison entre les besoins exprimés et l'application finale avant la mise en production.

2.3 Cycle de vie du projet (modèle en V) :

Le modèle en V est une approche structurée du développement logiciel qui organise les étapes du projet en deux parties :

La branche descendante (de gauche à droite) qui correspond à l'analyse et à la conception.

La branche montante (de droite à gauche) qui correspond aux différentes phases de test et de validation.

Chaque étape de la partie gauche a une étape de validation correspondante dans la partie droite, ce qui garantit la qualité et la conformité du produit.

1. Analyse des besoins

Cette étape consiste à identifier et recueillir les besoins fonctionnels et non fonctionnels. Dans le cadre de ce projet, les besoins portaient principalement sur la gestion des utilisateurs, la gestion des rôles et la gestion des applications.

Cette phase est validée par les tests d'acceptation, réalisés en fin de projet, qui permettent de vérifier que les attentes initiales de l'ADII sont bien respectées.

2. Spécifications des besoins

Une fois les besoins identifiés, ils sont traduits en spécifications détaillées. Celles-ci définissent les fonctionnalités de l'application (exemple : création de rôles, attribution d'applications, gestion des profils).

Cette phase est validée par les tests système, qui vérifient le respect des spécifications globales.

3. Conception

La conception définit l'architecture technique du projet. Elle inclut :

Le MCD (Modèle Conceptuel de Données) pour structurer la base de données.

Les diagrammes UML (cas d'utilisation, classes, séquences).

Les choix technologiques (Java Spring Boot, MySQL, etc.).

Cette phase est validée par les tests d'intégration, qui assurent que les différents modules fonctionnent correctement ensemble.

4. Implémentation (Codage)

C'est la phase de développement de l'application. Le code source est produit en suivant les spécifications et la conception définies. Ici, j'ai utilisé Spring Boot pour la partie back-end, MySQL pour la base de données, et des frameworks UML pour la conception.

Cette phase est validée par les tests unitaires, qui s'assurent du bon fonctionnement de chaque module indépendamment.

5. Vérification et validation (branche montante)

Tests unitaires : vérifient chaque composant isolé (par exemple, création d'un rôle).

Tests d'intégration : assurent la cohérence entre les composants (exemple : affectation d'un rôle à un utilisateur dans une application).

Tests système : valident que l'application complète fonctionne selon les spécifications.

Tests d'acceptation : réalisés par les utilisateurs ou les responsables fonctionnels, pour confirmer que le système répond aux besoins exprimés.

6. Maintenance

Enfin, après la mise en production, l'application entre en phase de maintenance. Cette étape vise à corriger d'éventuelles anomalies, à améliorer certaines fonctionnalités ou à adapter le système à de nouveaux besoins de l'ADII.

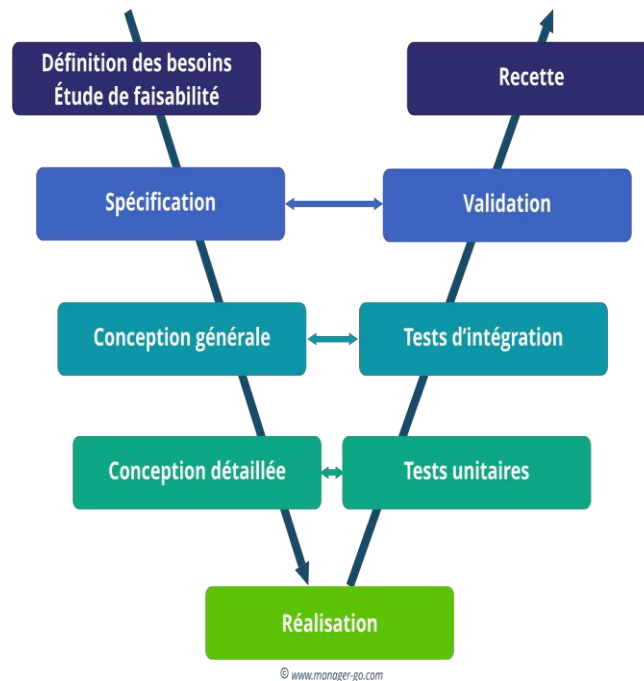


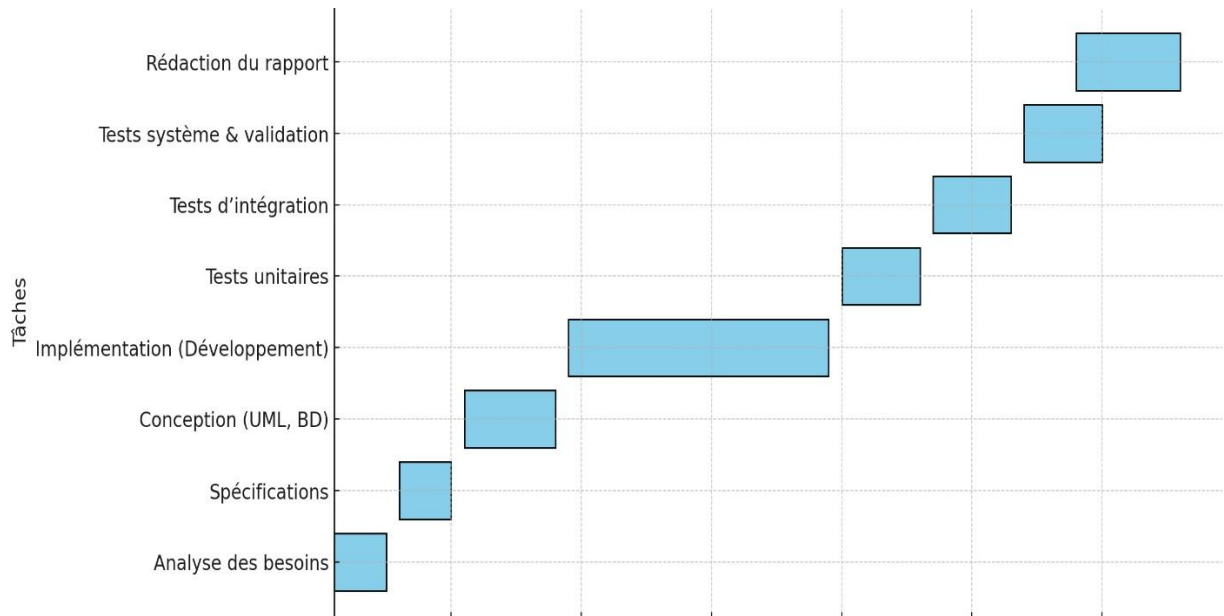
Figure 2.1 – cycle de vie

2.4 Conclusion et Diagramme de Gantt :

La méthodologie adoptée, basée sur le modèle en V, a permis de structurer le projet de manière claire et progressive. Elle a également assuré une correspondance directe entre les spécifications initiales et les tests réalisés, garantissant ainsi la fiabilité du produit développé.

La planification du stage a été représentée par un diagramme de Gantt, qui montre la répartition des tâches depuis l'étude des besoins jusqu'à la rédaction du rapport.

Figure 2.4 – diagramme de Gantt



2.5 Conclusion :

La démarche adoptée pour le développement de ce projet au sein de l'ADII a permis de structurer l'ensemble du travail de manière claire et progressive. En s'appuyant sur le modèle en V, chaque étape – de l'analyse des besoins à la mise en œuvre – a été reliée à une phase de validation correspondante, garantissant ainsi la conformité du produit aux attentes exprimées.

L'utilisation de cette méthodologie a favorisé la rigueur, la traçabilité et la qualité du développement, tout en réduisant les risques d'incohérence entre les spécifications et l'implémentation finale.

Par ailleurs, la planification à l'aide d'un diagramme de Gantt a permis de répartir efficacement les tâches sur la période de stage (du 29 juillet au 5 septembre 2025). Cette organisation a offert une vision claire de l'avancement du projet et assuré le respect des délais fixés.

Ainsi, le Chapitre II a mis en évidence l'importance de la méthodologie et de la planification dans la réussite d'un projet informatique, et constitue une base solide pour passer à l'analyse détaillée des besoins, présentée dans le chapitre suivant.

3 Chapitre III : Analyse des besoins

3.1 Introduction :

Toute conception logicielle débute par une analyse précise des besoins. Cette étape est fondamentale car elle permet de comprendre les attentes des utilisateurs, de définir les fonctionnalités à développer et d'identifier les contraintes techniques.

Dans le cadre de ce projet réalisé au sein de l'ADII, l'analyse des besoins avait pour objectif de répondre à une problématique bien précise : assurer une gestion centralisée, sécurisée et efficace des profils utilisateurs au sein des applications web internes.

Cette analyse s'est déroulée en deux volets :

Le recueil des besoins fonctionnels, qui concernent les fonctionnalités attendues par l'application.

Le recueil des besoins non fonctionnels, qui concernent la qualité du système (sécurité, performance, ergonomie, etc.).

Enfin, la modélisation des processus métiers a permis de traduire les besoins identifiés en schémas clairs, facilitant la compréhension et la conception de l'application.

3.2 Recueil des Besoins Fonctionnels :

Les besoins fonctionnels correspondent aux actions que le système doit permettre à ses utilisateurs d'effectuer. Dans ce projet, deux profils principaux sont concernés :

l'administrateur et le gestionnaire.

a) Côté Administrateur

Gestion des rôles :

Créer un rôle.

Modifier un rôle.

Supprimer un rôle.

Gestion des applications :

Créer une nouvelle application.

Modifier ou supprimer une application existante.

Associer des rôles spécifiques à chaque application.

Gestion des utilisateurs :

Afficher la liste des utilisateurs.

Attribuer le rôle de gestionnaire à certains utilisateurs.

Affecter à un gestionnaire les applications et les rôles correspondants.

b) Côté Gestionnaire

Attributions :

Sélectionner un utilisateur normal.

Lui attribuer une application parmi celles définies par l'administrateur.

Lui associer un ou plusieurs rôles disponibles dans cette application.

Utilisateurs :

Consulter la liste des utilisateurs normaux (hors administrateurs et gestionnaires).

Applications et rôles :

Visualiser les applications et leurs rôles définis par l'administrateur.

Ces fonctionnalités garantissent une gestion claire et hiérarchisée des accès aux applications de l'ADII.

3.3 Recueil des Besoins Non Fonctionnels :

Au-delà des fonctionnalités, l'application doit répondre à des critères de qualité afin d'être utilisable et efficace dans le contexte d'une administration telle que l'ADII.

Sécurité :

Authentification centralisée (via Active Directory).

Respect des droits d'accès selon les rôles définis.

Protection des données sensibles (cryptage des mots de passe, sécurisation des sessions).

Performance :

Réponse rapide aux requêtes, même avec un grand nombre d'utilisateurs.

Optimisation des requêtes SQL et de la gestion des sessions.

Ergonomie et accessibilité :

Interface intuitive pour faciliter la prise en main.

Navigation fluide entre les sections des tableaux de bord.

Scalabilité et maintenance :

Capacité à intégrer de nouvelles applications et de nouveaux rôles sans remettre en cause l'architecture globale.

Possibilité de corriger et d'améliorer facilement le système.

Interopérabilité :

Compatibilité avec l'infrastructure informatique de l'ADII (Wildfly/JBoss, Active Directory).

3.4 Modélisation des Processus Métiers :

Afin de mieux comprendre le fonctionnement du système et d'illustrer les interactions entre les différents acteurs, une modélisation a été réalisée à l'aide de diagrammes UML.

Diagramme de cas d'utilisation (Use Case) : représente les différentes interactions entre les acteurs (administrateur, gestionnaire, utilisateur) et le système.

Diagramme de classes : structure les entités principales (Utilisateur, Rôle, Application, Gestionnaire, Administrateur) et leurs relations.

MCD (Modèle Conceptuel de Données) : illustre l'organisation des données dans la base MySQL.

Ces modèles constituent une base solide pour la conception technique de l'application et garantissent que les besoins exprimés soient correctement traduits dans la solution finale.

3.5 Modèle Conceptuel de Données (MCD) :

Le Modèle Conceptuel de Données (MCD) permet de représenter l'organisation des données et les relations entre elles pour l'application de gestion des profils utilisateurs.

Dans notre projet, le MCD repose sur **5 tables principales** :

User (Utilisateur)

App (Application)

Role (Rôle)

RoleApp (*Association Role ↔ Application*)

UserRole (*Association User ↔ RoleApp*) Relations

principales :

Un utilisateur peut se voir attribuer plusieurs rôles dans différentes applications.

Un rôle peut être associé à plusieurs applications.

Les associations RoleApp et UserRole assurent une gestion hiérarchique et sécurisée des droits.

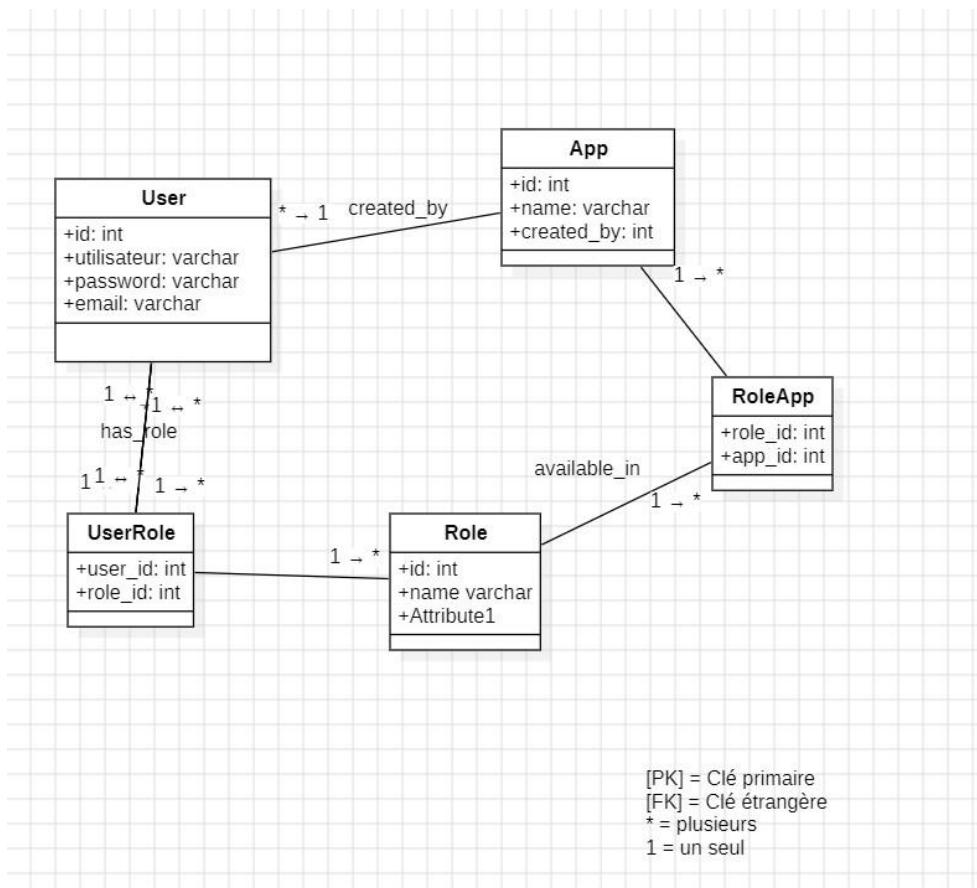


Figure 3.5 – Modèle conceptuel de données (MCD)

Ce modèle garantit une traçabilité complète des attributions, une flexibilité pour ajouter de nouvelles applications ou rôles, et une gestion sécurisée des droits d'accès.

3.6 Conclusion :

Le chapitre III a permis de formaliser les besoins et la structure des données de l'application :

Les besoins fonctionnels ont été clairement identifiés pour les administrateurs et les gestionnaires, assurant que toutes les actions nécessaires soient couvertes.

Les besoins non fonctionnels ont précisé les contraintes de sécurité, performance, ergonomie et scalabilité indispensables pour un système destiné à une administration publique.

Le MCD, basé sur les cinq tables User, App, Role, RoleApp et UserRole, a traduit ces besoins en une structure de données cohérente et évolutive.

Ainsi, ce chapitre fournit les bases solides nécessaires à la phase suivante : la conception technique de l'application, détaillée dans le chapitre IV.

4 Chapitre IV : Conception de l'application :

La conception de l'application constitue une étape essentielle après l'analyse des besoins et la modélisation des données. Elle permet de transformer les spécifications fonctionnelles et techniques en solutions concrètes, prêtes à être développées. Dans notre projet de gestion des profils utilisateurs, cette phase inclut l'architecture globale, la conception de la base de données, les interfaces utilisateur et les diagrammes UML.

4.1 Architecture générale de l'application :

L'architecture de l'application repose sur un modèle **client-serveur** moderne, combinant une couche frontend réactive et un backend robuste :

4.1.1 Côté Backend :

- Technologie : Java Spring Boot MVC.
- Gestion des dépendances : Gradle.
- Fonctionnalités principales :

Implémentation de la logique métier.

Gestion des rôles, applications et utilisateurs.

Services REST sécurisés pour le frontend.

Authentification et autorisation via JWT (JSON Web Token).

- Avantages : modularité, sécurité, facilité de maintenance et intégration avec Active Directory.

4.1.2 Base de données :

- Technologie : MySQL.
- Accès à la base : via MySQL Workbench ou le console intégré d'IntelliJ IDEA Ultimate.
- Rôle : stockage des utilisateurs, rôles, applications et attributions.
- Avantages : fiabilité, intégrité des données et support des relations complexes N:M via tables associatives (RoleApp, UserRole , UserApp).

4.1.3 Côté Frontend :

- Technologie : React.js + JavaScript.
- Fonctionnalité : affichage dynamique des dashboards, gestion des interactions utilisateurs, envoi et réception des requêtes API vers le backend.
- Avantages : réactivité, interface utilisateur fluide et modulaire, responsive design.

4.1.4 Sécurité et authentification :

Authentification centralisée : via Microsoft Active Directory.

Autorisation : basée sur les rôles attribués, vérifiée côté backend via JWT.

Protection des données : gestion des sessions sécurisées, cryptage des mots de passe.

Schéma global de l'architecture :

Client (React.js) ↔ Backend (Spring Boot MVC + JWT) ↔ Base de données MySQL.

Les dashboards Admin et Gestionnaire communiquent avec le backend via API sécurisées.

4.2 Conception de la base de données (schéma relationnel) :

La base de données est directement dérivée du MCD du chapitre précédent. Elle comprend 6 tables principales :

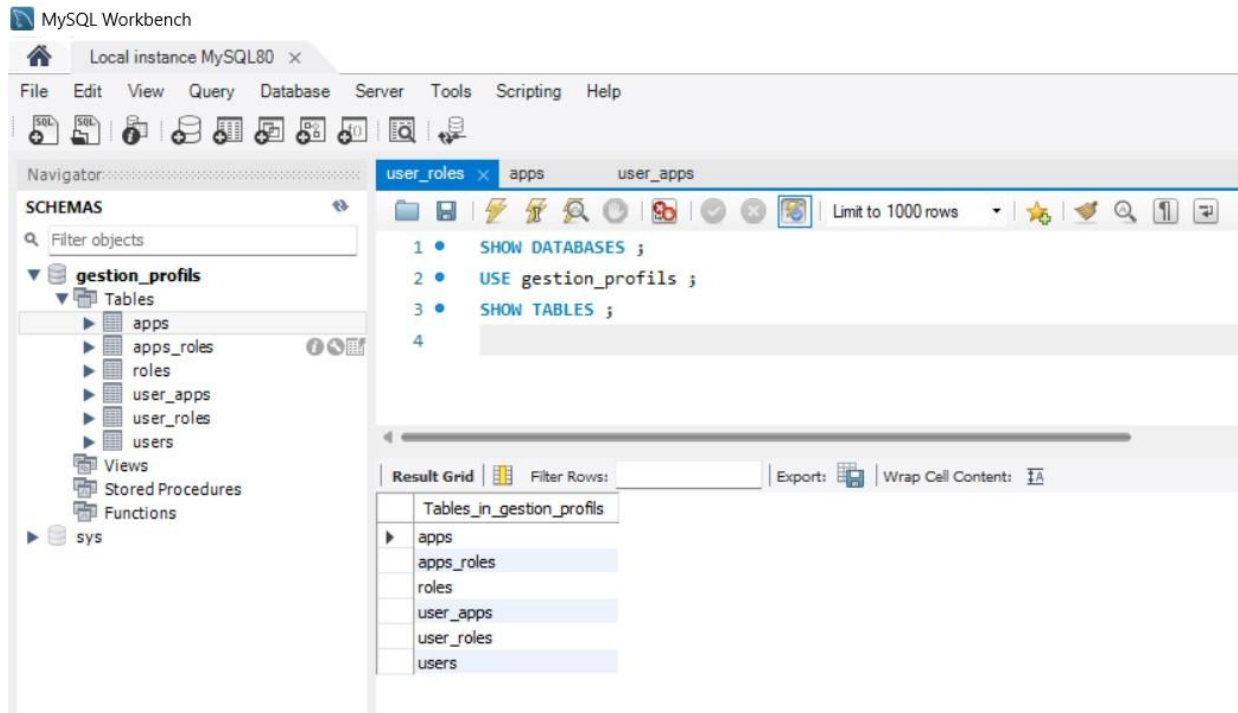


Figure 4.2 – schéma la base de données

Accès et manipulation des données :

MySQL Workbench : interface graphique pour gérer la base, créer des tables et exécuter des requêtes SQL.

Console IntelliJ IDEA Ultimate : accès direct à la base depuis l'IDE pour le développement et les tests.

4.3 Conception des interfaces utilisateurs

L'application dispose de deux dashboards principaux, conçus pour être intuitifs, clairs et ergonomiques :

+ Dashboard Administrateur

Gestion des rôles : créer, modifier, supprimer des rôles.

Gestion des applications : créer, modifier, supprimer des applications et leur associer des rôles.

Gestion des utilisateurs : consulter la liste des utilisateurs, attribuer des rôles de gestionnaire et des applications.

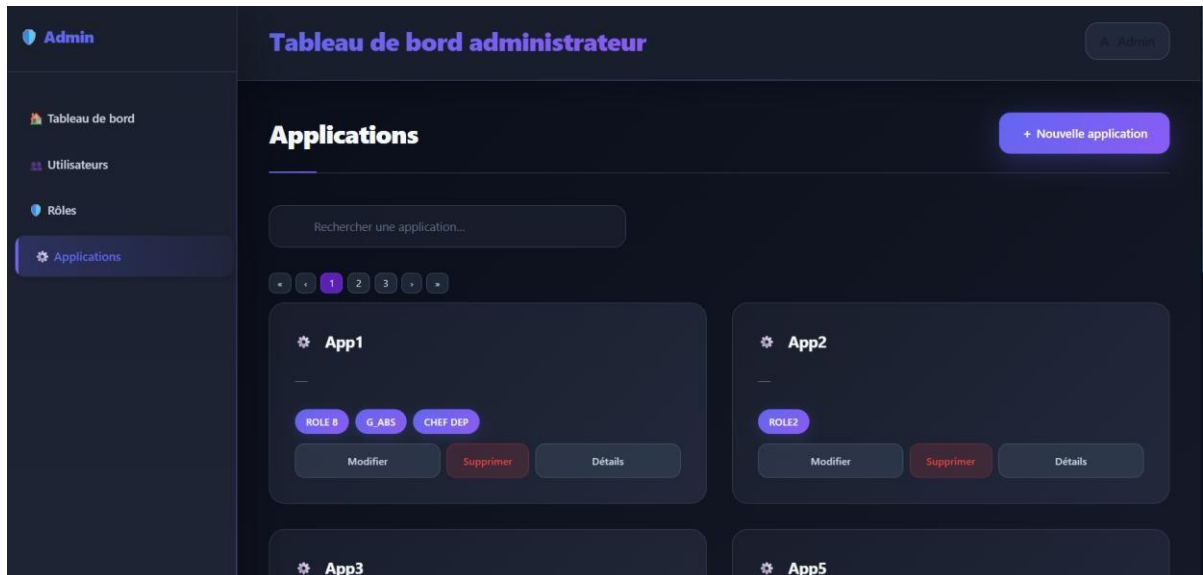


Figure 4.3 – interface admin

+ Dashboard Gestionnaire

Attributions : sélectionner un utilisateur normal et lui attribuer des rôles dans des applications définies par l'administrateur.

Utilisateurs : affichage des utilisateurs normaux uniquement.

Applications et rôles : consultation des applications et rôles définis par l'administrateur.

Principes de conception appliqués :

Navigation hiérarchisée via le menu latéral (side bar).

Validation côté client et côté serveur pour garantir l'intégrité des données.

Composants réactifs pour des mises à jour instantanées.

Design responsive pour différents appareils.

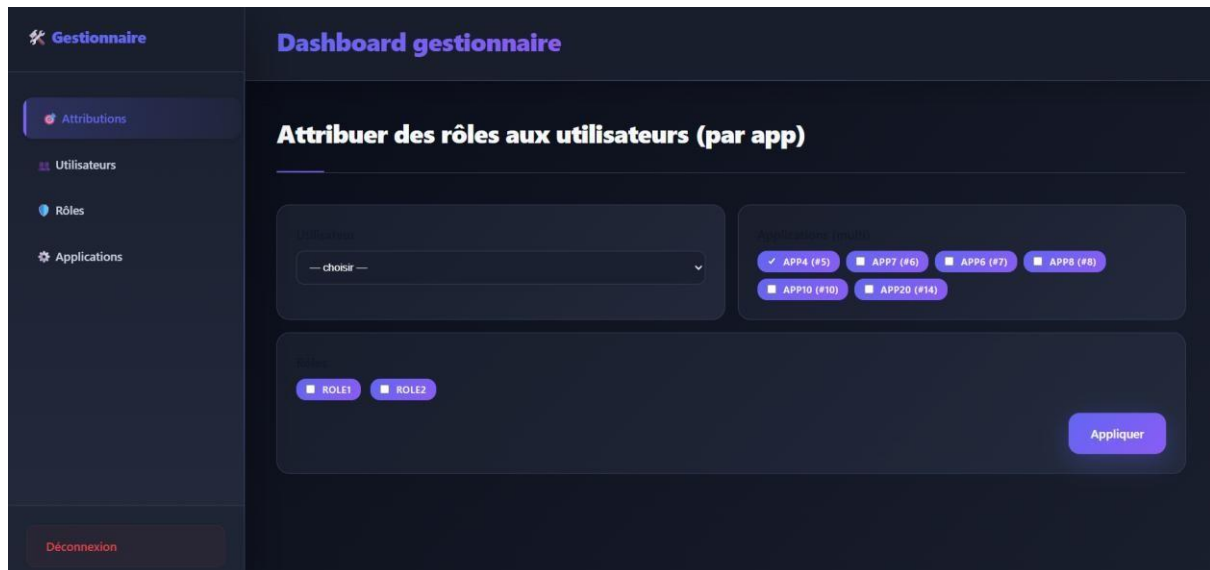


Figure 4.3 – interface gestionnaire

4.4 Diagrammes UML :

Les interfaces utilisateur doivent être conçues pour offrir une expérience fluide et intuitive :

4.4.1 Diagramme de cas d'utilisation

Administrateur : gérer les rôles, applications, utilisateurs et attribuer des droits aux gestionnaires.

Gestionnaire : attribuer des rôles aux utilisateurs normaux et consulter les applications et utilisateurs.

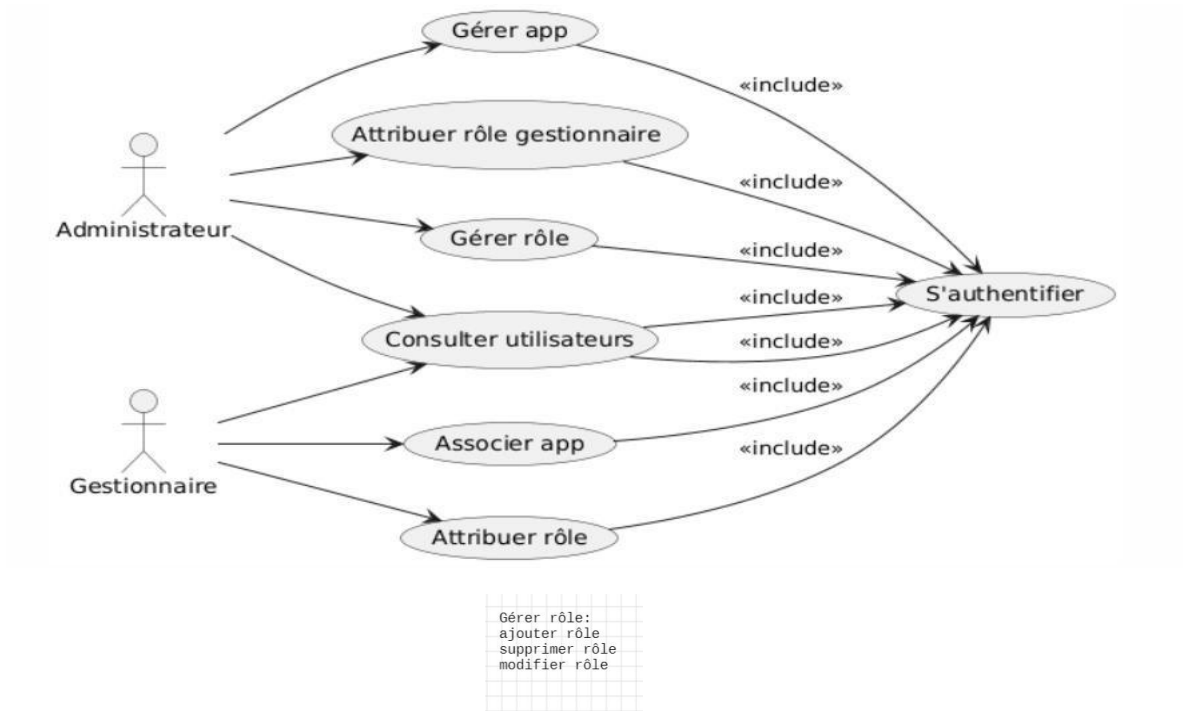


Figure 4.4.1 – Diagramme de cas d'utilisation

4.4.2 Diagramme de classes

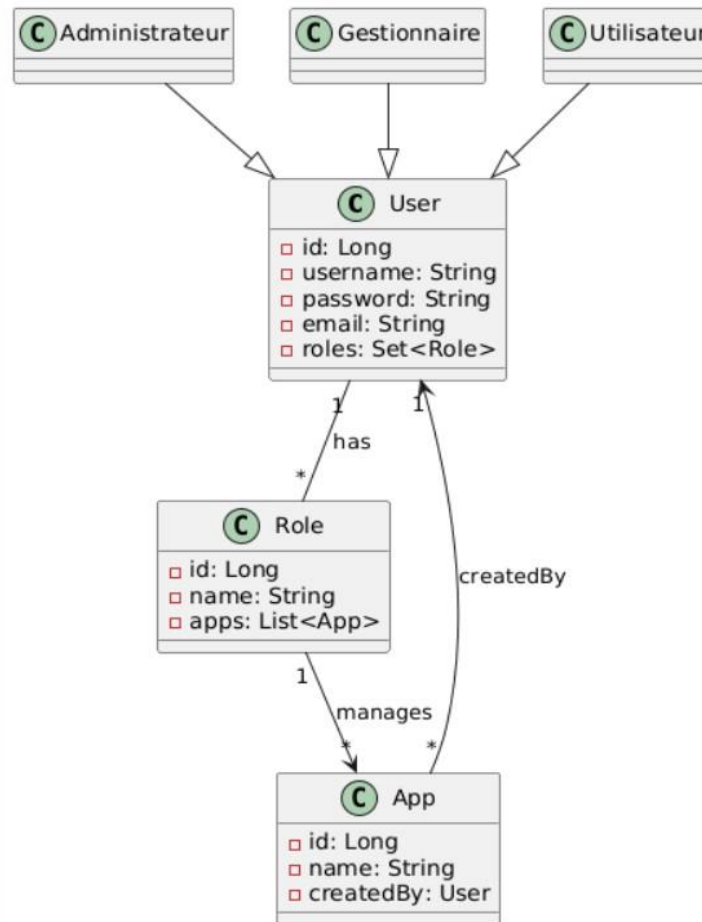


Figure 4.4.2 – Diagramme de classe

4.5 Conclusion

La conception de l'application a permis de définir :

Une architecture sécurisée et performante avec Spring Boot MVC, JWT et React.js.

Une base de données relationnelle robuste avec MySQL et accès via Workbench ou IntelliJ.

Des interfaces utilisateur ergonomiques pour les deux dashboards principaux.

Des modèles UML clarifiant les interactions et la structure du système.

Cette phase garantit que l'application est prête pour la phase de réalisation et de déploiement, détaillée dans le chapitre V.

5 Chapitre V : Réalisation et mise en œuvre :

La phase de réalisation est cruciale dans le développement logiciel, car elle consiste à transformer la conception en une application réellement fonctionnelle. Dans le cadre de ce projet, cette étape a permis de concrétiser les besoins identifiés, de développer les fonctionnalités prévues et de préparer l'application à son déploiement dans l'environnement professionnel de l'ADII.

5.1 Environnement de développement

Pour garantir la qualité, la sécurité et la performance de l'application, un environnement de développement moderne et robuste a été mis en place.

5.1.1 Backend : Spring Boot (Java)



Figure 5.1.1 – Java spring boot

Le backend constitue le cœur de l'application et gère la logique métier ainsi que l'accès aux données.

Technologie : Java Spring Boot MVC.

Gestion des dépendances : Gradle.

Rôle :

Implémentation de toutes les règles métiers liées aux utilisateurs, rôles et applications.

Fourniture de services REST sécurisés pour le frontend.

Gestion de l'authentification et de l'autorisation via JWT (JSON Web Token).

Avantages : architecture modulaire et maintenable, intégration facile avec Active Directory, sécurisation des échanges et capacité à évoluer avec de nouvelles fonctionnalités.

5.1.2 Frontend : React.js et JavaScript :

Le frontend assure l'interface utilisateur et la communication avec le backend.

Technologie : React.js combiné avec JavaScript.

Fonctionnalités :

Construction dynamique des dashboards Administrateur et Gestionnaire.

Interaction fluide avec le backend via API REST.

Mise à jour en temps réel des informations affichées.

Avantages : interface moderne, intuitive, réactive et compatible avec différents appareils (responsive design).

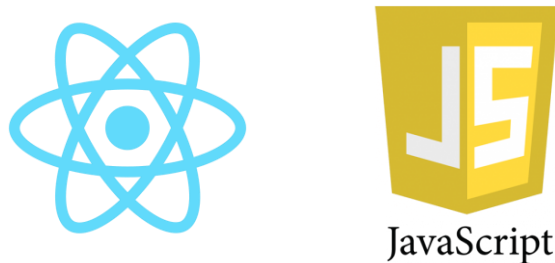


Figure 5.1.2 – React.js et JavaScript

5.2 Base de données : MySQL

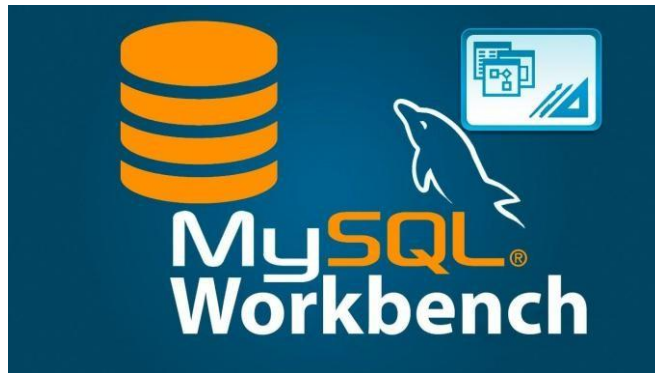


Figure 5.2 – MySQL Workbench

- Technologie : MySQL
- Rôle : stockage des utilisateurs, applications, rôles et attributions.
- Accès et manipulation :

MySQL Workbench pour la gestion graphique des tables et des requêtes.

Console intégrée dans IntelliJ IDEA Ultimate pour les tests et le développement direct.

- Avantages : fiabilité, support des relations complexes N:M via tables associatives (UserApp, UserRole, RoleApp), et intégrité des données.

5.2.4 Outils de conception :

- **StarUML** : pour la création des diagrammes UML (cas d'utilisation, classes).
- **PlantUML** : pour générer des diagrammes dynamiques et intégrés dans la documentation.



5.2 Implémentation des modules fonctionnels :

La réalisation de l'application repose sur un environnement de développement bien défini, comprenant plusieurs technologies et outils :

5.2.1 Module gestion des rôles :

- Permet à l'administrateur de créer, modifier et supprimer des rôles.
- Validation côté serveur pour éviter les doublons et garantir l'intégrité des données.
- Les rôles créés sont immédiatement disponibles pour l'attribution dans les applications.

5.2.2 Module gestion des applications :

- Création, modification et suppression des applications par l'administrateur.
- Association des rôles existants à chaque application.
- Mise à jour dynamique des applications disponibles pour les gestionnaires et utilisateurs.

5.2.3 Module gestion des utilisateurs :

- Consultation de la liste des utilisateurs.
- Attribution du rôle de gestionnaire et des applications associées par l'administrateur.

- Les gestionnaires peuvent ensuite attribuer des rôles aux utilisateurs normaux à partir des applications disponibles

5.3 Interfaces de l'application :

Dashboard Administrateur : navigation via un menu latéral avec trois sections principales (Rôles, Applications, Utilisateurs).

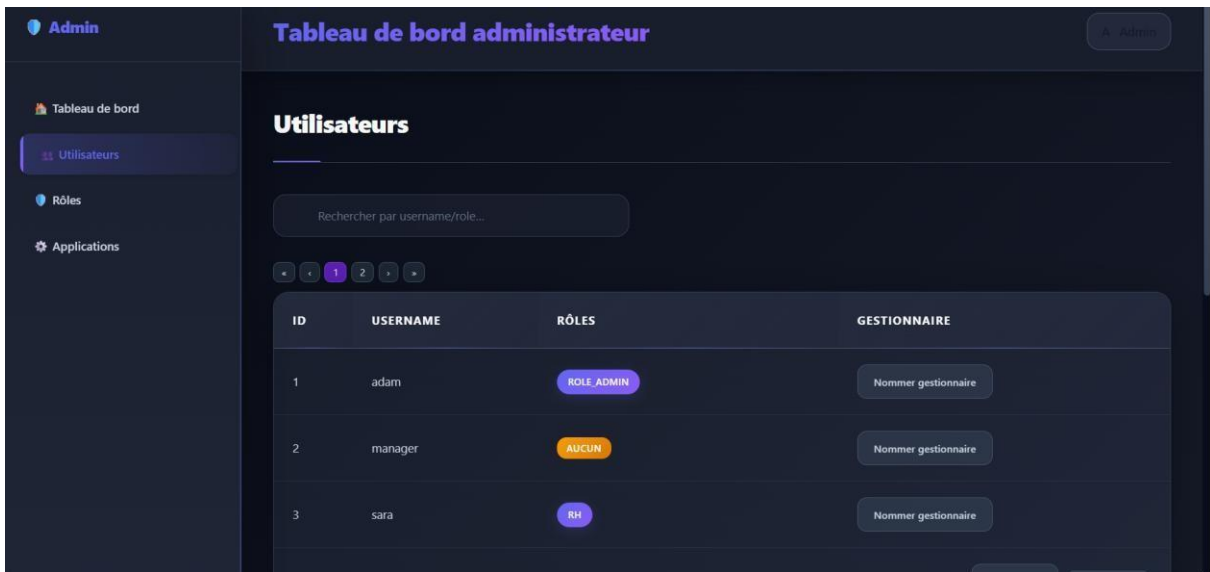
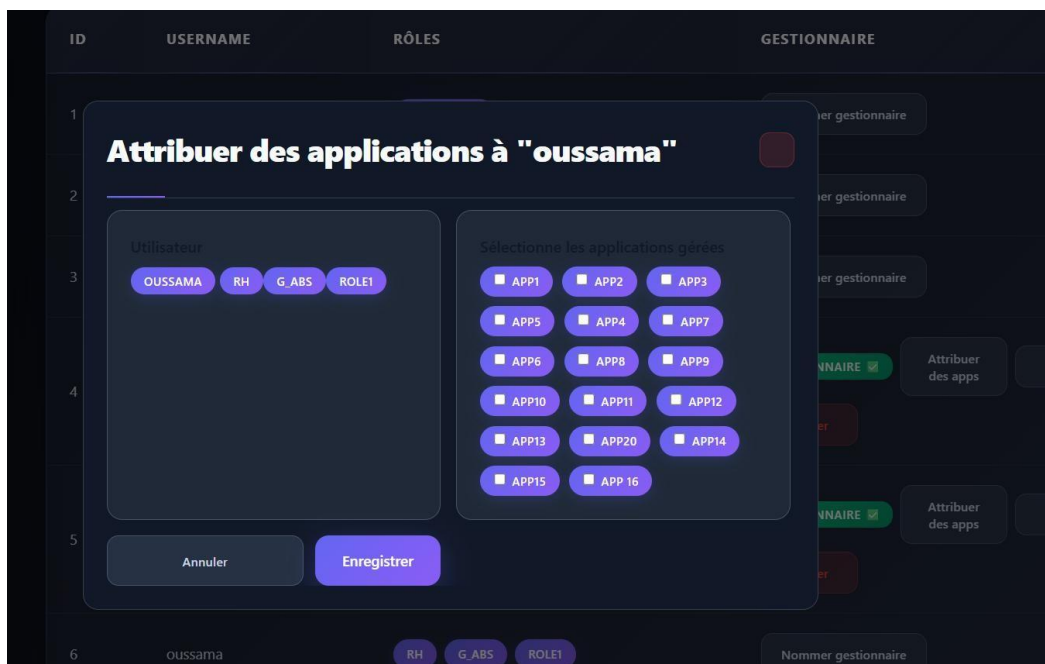


Figure 5.3 – Dashboardadmin (utilisateur)



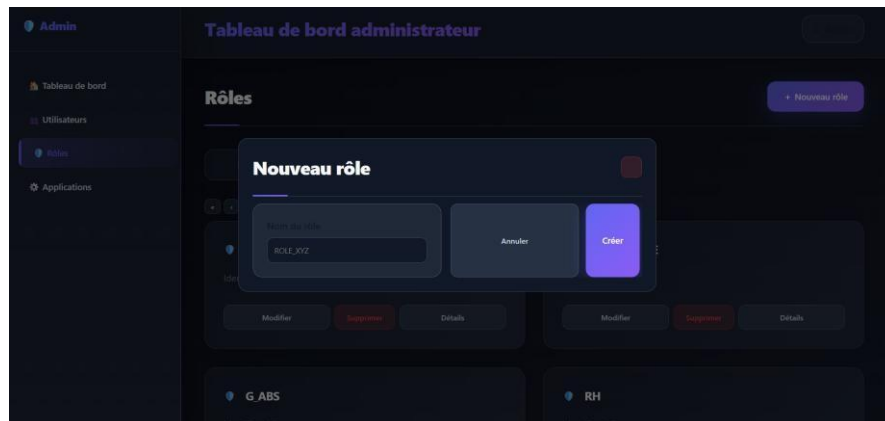


Figure 5.3 – Dashboardadmin (Role)

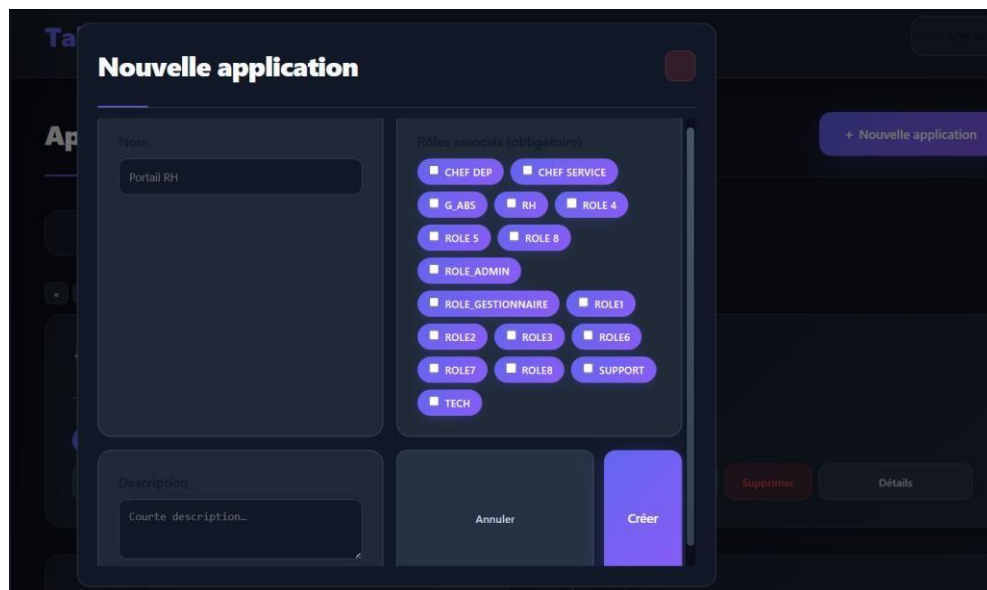
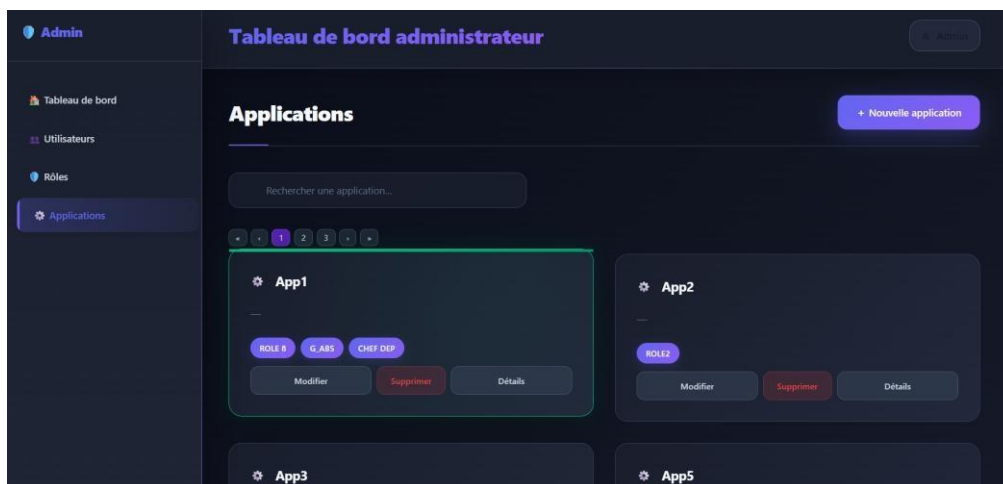


Figure 5.3 – Dashboardadmin (App)

- Dashboard Gestionnaire : accès aux attributions, consultation des utilisateurs normaux, et visualisation des applications et rôles définis par l'administrateur.
- Composants réactifs pour un rendu instantané des modifications.
- Design responsive pour garantir l'accessibilité sur différents appareils.

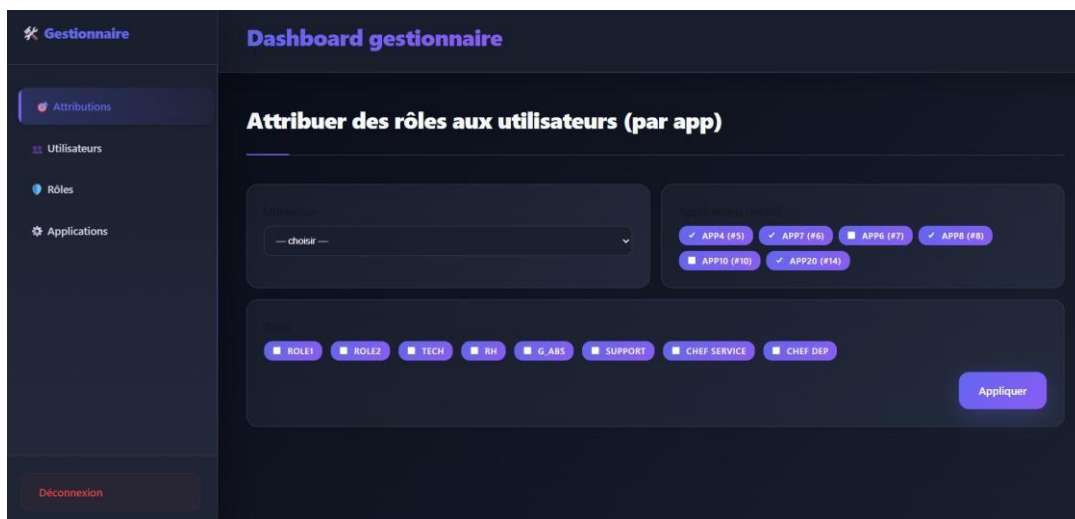
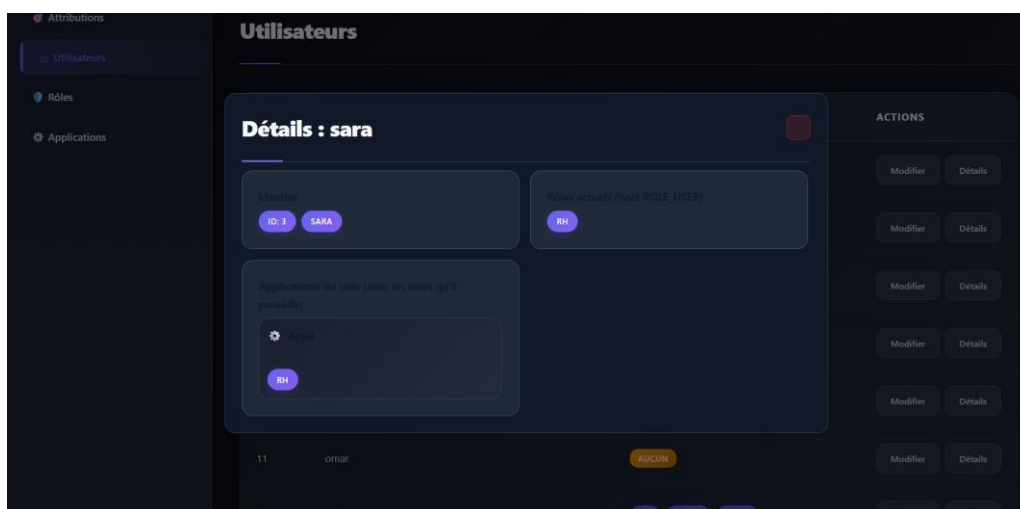


Figure 5.3 – Dashboardgestionnaire (Attribution)



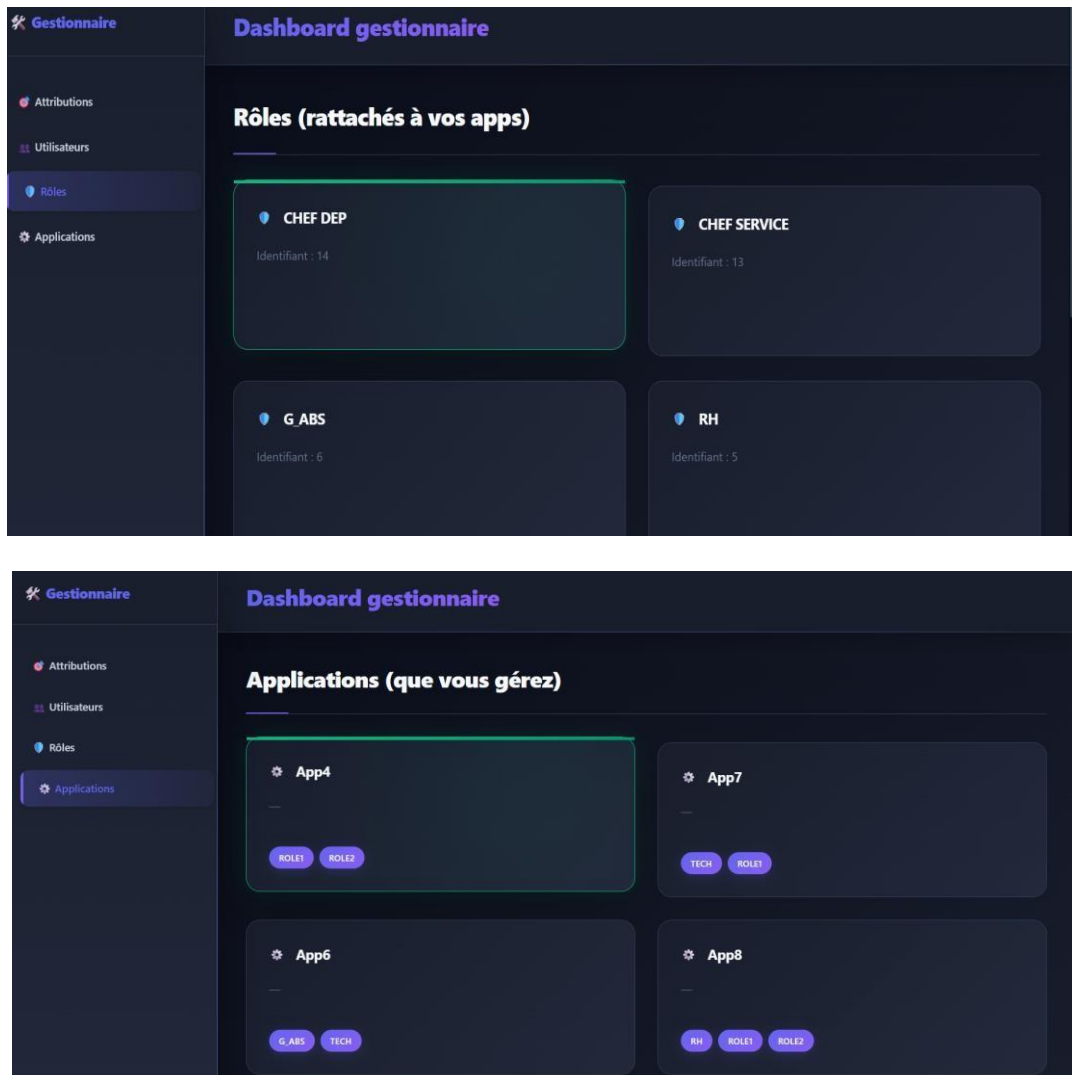


Figure 5.3 – Dashboardgestionnaire (rôles et app)

5.5 Conclusion :

La phase de réalisation et de mise en œuvre a permis de :

Développer une application fonctionnelle et sécurisée conforme aux besoins identifiés.

Intégrer de manière efficace le backend Spring Boot, le frontend React.js et la base de données MySQL.

Mettre en place une interface intuitive pour les dashboards Administrateur et Gestionnaire.

Déployer l'application dans un environnement professionnel, utilisant Active Directory et Wildfly.

Cette étape clôt la réalisation technique et prépare l'application à un usage opérationnel dans le cadre de la gestion des profils utilisateurs à l'ADII.

5.6 Conclusion générale :

Le stage effectué au sein de l'Administration des Douanes et Impôts Indirects (ADII) de Rabat a été une expérience enrichissante, tant sur le plan technique que professionnel. Il m'a permis de concevoir et de développer une application de gestion des profils utilisateurs, répondant aux besoins de centralisation et de hiérarchisation des accès aux applications internes.

Durant ce stage, j'ai consolidé mes compétences en backend avec Spring Boot, en frontend avec React.js, et en gestion de base de données MySQL. J'ai pu mettre en pratique les concepts de la programmation orientée objet, de la création d'API REST et de la modélisation des données, tout en respectant les besoins fonctionnels et non fonctionnels définis.

Ce projet m'a offert une vision complète de la conduite d'un projet logiciel, depuis l'analyse des besoins jusqu'à la réalisation concrète des modules fonctionnels, et m'a sensibilisé à l'importance de la sécurité, de l'ergonomie et de la maintenance dans le développement d'applications web.

Bien que l'application réponde aux besoins identifiés, elle pourra évoluer dans le futur avec de nouvelles fonctionnalités, comme l'extension des rôles ou la gestion d'applications supplémentaires.

En somme, ce stage a été une étape déterminante dans mon parcours, me permettant de transformer mes connaissances théoriques en solutions concrètes et fonctionnelles, et de me préparer à relever de futurs défis dans le développement d'applications web.

5.7 Bibliographie :

Spring Boot Documentation. <https://spring.io/projects/spring-boot>

React.js Documentation. <https://reactjs.org/docs/getting-started.html>

MySQL Documentation. <https://dev.mysql.com/doc/>

JWT (JSON Web Token) Introduction. <https://jwt.io/introduction>

Baeldung. (2021). *Guide to Spring Boot Security and JWT*. <https://www.baeldung.com/springsecurity-jwt>

FreeCodeCamp. (2020). *React.js Tutorial – Full Course for Beginners*. <https://www.freecodecamp.org/news/react-js-tutorial-for-beginners/>

W3Schools. (2023). *MySQL Tutorial*. <https://www.w3schools.com/mysql/>

StarUML. *Software Modeling Tool*. <https://staruml.io/>

PlantUML. *Open-source UML tool*. <https://plantuml.com/>
